

10 Agentic RAG Interview Questions & Answers

CRAG | Self-RAG | Adaptive RAG | Reflection | GraphRAG | Multi-Index | 2026

Query Rewriting | Self-Correction | Conflict Resolution | LangGraph

CRAG	Self-RAG	Adaptive RAG
Self-Correction	Reflection	Query Rewriting
GraphRAG	Multi-Index	Evaluation
	Conflict Resolution	

AmanAI Lab

amanailab.com | YouTube: @AmanAILab

FREE — github.com/amanailab/Production-level-Generative-AI-Interview-Questions

Table of Contents

■ Agentic RAG Architecture

- Q1. What is CRAG (Corrective RAG) and how does it improve on standard Agentic RAG?
- Q2. What is Self-RAG and how does it differ from standard RAG and CRAG?
- Q3. How does Adaptive RAG dynamically choose between different retrieval strategies?

■ Self-Correction & Reflection

- Q4. How do you implement a self-correcting RAG loop where the agent retries on poor quality answers?
- Q5. What is Reflection in Agentic RAG and how is it implemented using LangGraph?
- Q6. How do you implement query rewriting in Agentic RAG to improve retrieval quality?

■ Multi-Index & Graph RAG

- Q7. What is GraphRAG and when does it outperform standard vector RAG?
- Q8. How do you implement multi-index RAG where different questions route to different knowledge bases?

■ Production & Evaluation

- Q9. How do you evaluate Agentic RAG systems beyond standard RAGAS metrics?
- Q10. How do you handle knowledge conflicts in Agentic RAG when retrieved documents contradict each other?

Agentic RAG Architecture

Q1

What is CRAG (Corrective RAG) and how does it improve on standard Agentic RAG?

Answer:

CRAG (Corrective Retrieval-Augmented Generation) adds a self-evaluation step that standard Agentic RAG lacks. In standard Agentic RAG, the agent retrieves chunks and generates — even if retrieved chunks are irrelevant. CRAG adds a **retrieval evaluator** between retrieval and generation that scores chunk relevance. Three outcomes: (1) **Correct** (score high) — chunks are relevant, proceed to generation. (2) **Incorrect** (score low) — chunks are irrelevant, discard and fall back to web search. (3) **Ambiguous** (score medium) — some chunks relevant, some not. Filter relevant chunks and supplement with web search. Key innovation: the agent **KNOWS** when its retrieval failed and corrects it automatically instead of generating a hallucinated answer from bad context.

■ **Example:**

CRAG in action for a product support RAG:

User: 'How do I configure OAuth2 in TechCorp API v3.5?'

Step 1: Standard retrieval

Retrieved 5 chunks from knowledge base

Scores: [0.91, 0.88, 0.72, 0.41, 0.38]

Step 2: CRAG evaluator scores each chunk:

Chunk 1 (OAuth2 v3.5 guide): CORRECT -> keep

Chunk 2 (OAuth2 general): CORRECT -> keep

Chunk 3 (Auth overview): AMBIGUOUS -> keep, flag

Chunk 4 (v2.1 OAuth guide): INCORRECT -> discard (wrong version!)

Chunk 5 (unrelated billing): INCORRECT -> discard

Step 3: Web search supplement (because chunks 4,5 discarded)

Search: 'TechCorp API v3.5 OAuth2 configuration 2026'

Found: latest official docs with v3.5 specifics

Step 4: Generate from:

2 correct chunks + 1 ambiguous chunk + web search results

Answer: accurate, version-specific, no hallucination

Without CRAG: would have used wrong v2.1 OAuth guide

-> given outdated, incorrect configuration steps

With CRAG: caught the version mismatch, corrected automatically

■ *Interview Tip: Say 'CRAG adds a quality gate between retrieval and generation. The evaluator acts like a human reviewer — it checks if retrieved chunks actually answer the question before passing them to the LLM. Without this gate, bad retrieval silently produces hallucinated answers.' The quality gate framing is memorable.*

Q2

What is Self-RAG and how does it differ from standard RAG and CRAG?

Answer:

Self-RAG trains the LLM itself to decide when to retrieve, what to retrieve, and how to use retrieved content — using special reflection tokens. Unlike standard RAG (always retrieves) or Agentic RAG (agent decides), Self-RAG is baked INTO the model. Four special tokens: (1) **[Retrieve]** — model decides retrieval is needed. (2) **[IsRel]** — model judges if retrieved chunk is relevant. (3) **[IsSup]** — model judges if its generation is supported by the chunk. (4) **[IsUse]** — model judges if the response is useful to the user. Key advantage: no separate orchestration layer needed. The model itself handles all retrieval decisions. Key limitation: requires fine-tuning — can't use with GPT-4 or Claude directly. Used with open-source models (Llama). In practice: Self-RAG is powerful for specialized domains where you can afford fine-tuning.

■ Example:

Self-RAG vs standard RAG comparison:

Query: 'What is the capital of France?'

Standard RAG (always retrieves):

-> Retrieves documents about France

-> Generates: 'Paris' (wasted retrieval for known fact!)

Cost: 1 embedding + vector search + LLM call

Self-RAG:

Model generates [No Retrieve] token

-> Directly answers: 'Paris' (model already knows this!)

Cost: just the LLM call (no retrieval!)

Query: 'What is our Q3 2026 revenue breakdown?'

Standard RAG (always retrieves):

-> Retrieves financial documents

-> Generates answer from context

Self-RAG:

Model generates [Retrieve] token (doesn't know this!)

-> Retrieves financial documents

-> Generates draft answer

-> Generates [IsSup] = YES (generation supported by chunks)

-> Generates [IsUse] = YES (answer is useful)

-> Returns final answer

Self-RAG saves unnecessary retrievals on simple queries
by ~40% compared to standard always-retrieve RAG.

■ *Interview Tip: The key insight about Self-RAG: 'Unlike standard RAG that always retrieves regardless of need, Self-RAG is trained to decide WHEN to retrieve. This eliminates wasted retrievals on factual questions the model already knows — saving cost and latency.' Show you understand the efficiency angle.*

Q3

How does Adaptive RAG dynamically choose between different retrieval strategies?

Answer:

Adaptive RAG uses a query classifier to route different query types to the most appropriate retrieval strategy — rather than using one fixed strategy for all queries. Three strategy levels: (1) **No retrieval** — simple factual queries the LLM already knows. 'What is the Pythagorean theorem?' No need to retrieve. (2) **Single-shot retrieval** — standard one-pass RAG. Most business knowledge queries fit here. (3) **Iterative retrieval** — complex multi-hop queries requiring multiple retrieval rounds. 'How did our Q3 strategy perform vs competitor X's announced plans?' The classifier routes each query to the cheapest strategy that can answer it. Adaptive RAG reduces cost and latency by avoiding expensive iterative retrieval for simple queries.

■ Example:

Adaptive RAG routing in a financial analyst assistant:

Query 1: 'What is compound interest?'

Classifier: SIMPLE FACTUAL

Strategy: No retrieval

LLM answers from training knowledge

Latency: 0.8s | Cost: \$0.001

Query 2: 'What was our revenue in Q3 2026?'

Classifier: SINGLE DOCUMENT LOOKUP

Strategy: Single-shot RAG

Retrieve from financial reports -> generate

Latency: 1.4s | Cost: \$0.004

Query 3: 'Compare our Q3 2026 margin vs industry average and explain the gap given our recent M&A; activity'

Classifier: MULTI-HOP COMPLEX

Strategy: Iterative retrieval

Hop 1: Retrieve our Q3 margins

Hop 2: Retrieve industry benchmarks

Hop 3: Retrieve M&A; announcement docs

Synthesize all three

Latency: 4.2s | Cost: \$0.018

Cost comparison without Adaptive RAG:

All 3 queries use iterative RAG

Total cost: $\$0.018 \times 3 = \0.054

With Adaptive RAG:

Total cost: $\$0.001 + \$0.004 + \$0.018 = \0.023

Cost reduction: 57% on this query set!

■ *Interview Tip: Give the cost comparison numbers. 'Adaptive RAG reduced our costs by 57% by routing simple queries to cheaper strategies. The classifier costs almost nothing — a small LLM or even a rule-based system — but saves expensive iterative retrieval for 60% of queries that don't need it.' Cost savings make this concrete.*

Self-Correction & Reflection

Q4

How do you implement a self-correcting RAG loop where the agent retries on poor quality answers?

Answer:

A **self-correcting RAG loop** evaluates the generated answer quality and automatically retries with a different strategy if quality is insufficient. Implementation: (1) **Generate initial answer** using retrieved context. (2) **Evaluate quality** — check faithfulness (is answer grounded in context?), relevancy (does it answer the question?), completeness (are all parts addressed?). (3) **If quality passes threshold** — return the answer. (4) **If quality fails** — diagnose the failure type and apply the appropriate correction: re-query with different terms, retrieve from different index, use web search as fallback, decompose into sub-questions. (5) **Max retry limit** — stop after 3 attempts to prevent infinite loops. Key insight: different failure modes need different corrections. Retrieval failure → new query. Faithfulness failure → stronger grounding prompt. Completeness failure → sub-question decomposition.

■ Example:

Self-correcting loop implementation:

```
class SelfCorrectingRAG:
    MAX_RETRIES = 3

    async def answer(self, query: str) -> str:
        attempt = 0
        strategy = 'standard'

        while attempt < self.MAX_RETRIES:
            # Step 1: Retrieve based on current strategy
            chunks = await self.retrieve(query, strategy)

            # Step 2: Generate answer
            answer = await self.generate(query, chunks)

            # Step 3: Evaluate quality
            scores = await self.evaluate(query, answer, chunks)

            # Step 4: Check if good enough
            if scores['faithfulness'] > 0.85 and \
                scores['relevancy'] > 0.80:
                return answer # SUCCESS

            # Step 5: Diagnose and correct
            if scores['faithfulness'] < 0.70:
                # LLM hallucinating -> stronger grounding
                strategy = 'strict_grounding'
            elif scores['relevancy'] < 0.70:
                # Wrong chunks retrieved -> rewrite query
                query = await self.rewrite_query(query)
                strategy = 'hybrid_search'
            else:
                # Partial answer -> decompose
                strategy = 'sub_question_decomposition'

            attempt += 1

        # Max retries hit -> return best attempt with caveat
        return answer + '\n[Note: Answer confidence is limited]'
```

■ *Interview Tip: Mention diagnosing failure TYPE before correcting. 'I never just retry with the same strategy — I diagnose WHY the answer failed first. Low faithfulness means the LLM is hallucinating despite good context, so I strengthen the grounding prompt. Low relevancy means bad chunks, so I rewrite the query. Different diagnoses need different treatments.'*

Q5

What is Reflection in Agentic RAG and how is it implemented using LangGraph?

Answer:

Reflection in Agentic RAG is a pattern where the agent critiques its own output and iteratively improves it before returning to the user. The agent acts as both the generator AND the critic. Process: (1) **Generate** — initial answer from retrieved context. (2) **Reflect** — a separate LLM call critiques the answer: What is missing? What is incorrect? What could be clearer? (3) **Revise** — rewrite the answer addressing the critique. (4) **Loop** — repeat until quality threshold met or max iterations reached. In LangGraph: implement as a cycle in the graph where the reflection node can loop back to the generation node. Use conditional edges to exit when quality is sufficient. Key consideration: reflection adds latency (1 extra LLM call per iteration) and cost. Only use when answer quality is critical (medical, legal, financial).

■ **Example:**

Reflection pattern in LangGraph:

```
from langgraph.graph import StateGraph, END

class RAGState(TypedDict):
    query: str
    context: list
    answer: str
    critique: str
    iteration: int
    quality_score: float

def generate(state: RAGState) -> RAGState:
    answer = llm.generate(state['query'], state['context'])
    return {**state, 'answer': answer}

def reflect(state: RAGState) -> RAGState:
    critique_prompt = f'''
Review this answer for accuracy, completeness, and clarity:
Question: {state["query"]}
Answer: {state["answer"]}
Context: {state["context"]}

Identify: (1) Missing information (2) Potential errors
(3) Clarity issues. Score quality 0-10.
Output JSON: {{"critique": "...", "score": X}}
'''
    result = llm.complete(critique_prompt)
    parsed = json.loads(result)
    return {**state,
            'critique': parsed['critique'],
            'quality_score': parsed['score'],
            'iteration': state['iteration'] + 1}

def revise(state: RAGState) -> RAGState:
    revised = llm.revise(state['answer'], state['critique'])
    return {**state, 'answer': revised}

def should_continue(state: RAGState) -> str:
    if state['quality_score'] >= 8.0: return END
    if state['iteration'] >= 3: return END # max iterations
    return 'revise'

graph = StateGraph(RAGState)
graph.add_node('generate', generate)
graph.add_node('reflect', reflect)
graph.add_node('revise', revise)
graph.add_edge('generate', 'reflect')
graph.add_conditional_edges('reflect', should_continue,
                             {'revise': 'revise', END: END})
graph.add_edge('revise', 'reflect') # loop back!
```

■ *Interview Tip: Mention when NOT to use reflection. 'Reflection improves quality but adds 1-2 LLM calls per iteration. For a customer FAQ chatbot, this is overkill. I use reflection only for high-stakes outputs — medical reports, legal documents, financial analysis — where the extra cost is justified by the quality requirement.'*

Q6

How do you implement query rewriting in Agentic RAG to improve retrieval quality?

Answer:

Query rewriting transforms the user's original query into a better search query before hitting the vector store. Why it matters: users write conversational queries ('what did we say last meeting about pricing?'), but vector stores retrieve better with declarative statements and rich keywords. Rewriting strategies: (1) **Keyword extraction** — extract core keywords, remove stop words. (2) **Query expansion** — add synonyms, related terms. 'What is churn?' → 'customer churn rate attrition retention loss'. (3) **Contextual rewrite** — use conversation history to resolve pronouns and references. 'What about that?' → 'What is the pricing strategy for the enterprise tier?'. (4) **Multi-query generation** — generate 3-5 different phrasings of the same question, retrieve for all, combine results. Increases recall significantly. (5) **HyDE** — generate a hypothetical answer, use that as the query embedding.

■ Example:

Multi-query rewriting in practice:

Original user query:

'Why is our conversion dropping?'

Problem: too vague for vector search

Vector search returns generic marketing docs

Step 1: Contextual resolution

(From conversation: user is VP Sales, discussing Q3)

Resolved: 'Why is sales conversion rate dropping in Q3 2026?'

Step 2: Multi-query generation (LLM creates 4 versions)

Q1: 'Q3 2026 sales conversion rate decline analysis'

Q2: 'Factors causing lower lead to customer conversion'

Q3: 'Sales funnel drop-off rates Q3 2026'

Q4: 'Revenue impact of conversion rate decrease 2026'

Step 3: Retrieve for all 4 queries

Q1 retrieves: sales performance report

Q2 retrieves: lead quality analysis doc

Q3 retrieves: funnel analytics dashboard notes

Q4 retrieves: CFO revenue forecast memo

Step 4: Deduplicate and rerank combined results

Final context: 4 highly relevant docs (vs 1 generic with original)

Step 5: Generate comprehensive answer

Answer covers: conversion data + root causes + revenue impact

Retrieval improvement:

Original query: context_precision = 0.61

Multi-query: context_precision = 0.89 (+46%)

■ *Interview Tip: Give the 46% improvement number. 'Multi-query retrieval improved our context precision from 0.61 to 0.89 — a 46% improvement — just by generating 4 phrasings of the same question. It costs 4x more retrieval operations but the quality gain is worth it for complex analytical queries.'*

Multi-Index & Graph RAG

Q7

What is GraphRAG and when does it outperform standard vector RAG?

Answer:

GraphRAG (Microsoft, 2024) represents the knowledge base as a graph of entities and relationships instead of (or in addition to) a flat vector index. Nodes = entities (people, places, concepts, products). Edges = relationships ('CEO of', 'acquired', 'competes with', 'caused by'). Why it outperforms standard RAG: (1) **Multi-hop reasoning** — 'How are Company A and Company B connected?' requires traversing

relationships, not just finding similar text. (2) **Global queries** — 'What are the main themes across ALL documents?' Vector RAG retrieves locally relevant chunks. GraphRAG can summarize at the graph community level. (3) **Relationship queries** — 'Who worked on the same projects as Alice?' impossible with vector RAG, trivial with graph traversal. When standard vector RAG is better: (a) Simple factual lookups. (b) Small document collections. (c) When relationships aren't the focus. GraphRAG is slower and more expensive to build but dramatically better for relationship-heavy queries.

■ Example:

GraphRAG vs Vector RAG on financial documents:

Document corpus: 5,000 SEC filings, earnings calls, news articles

Query 1: 'What did CEO John Smith say about margins?'

Vector RAG: WINS

Simple semantic search finds relevant quotes

Latency: 0.9s

Query 2: 'Which companies in our portfolio are exposed to the same supply chain risks as Company X?'

Vector RAG: FAILS

Can find supply chain text but can't traverse company-to-supplier relationships

Answer: incomplete and potentially wrong

GraphRAG: WINS

Graph nodes: Companies, Suppliers, Risk Factors

Graph edges: 'supplies_to', 'exposed_to', 'competes_with'

Traversal: Company X -> suppliers -> other companies using same suppliers -> return list

Answer: precise list of 7 portfolio companies

Latency: 2.1s

Query 3: 'Summarize the main themes across all Q3 2026 earnings calls'

Vector RAG: PARTIAL

Retrieves some relevant chunks, misses global patterns

GraphRAG Community Summaries: WINS

Pre-computed summaries at community level

'Top themes: AI investment, margin pressure, China exposure'

Comprehensive global view impossible with chunk retrieval

■ *Interview Tip: Mention the two GraphRAG modes. 'GraphRAG has two query modes: local search (like standard RAG but graph-aware) and global search (uses pre-computed community summaries for dataset-wide questions). The global search mode is what truly differentiates GraphRAG — it can answer questions about the entire document corpus that vector RAG simply cannot.'*

Q8

How do you implement multi-index RAG where different questions route to different knowledge bases?

Answer:

Multi-index RAG maintains separate vector indices for different knowledge domains and uses an intelligent router to send each query to the most relevant index (or multiple indices). Why separate indices: (1) Different domains have different optimal embedding models and chunking strategies. (2) Access control — some users should only access certain indices. (3) Freshness — some indices update daily (news), others rarely (policies). (4) Quality — mixing high-quality curated docs with raw data degrades retrieval quality. Router types: (1) **LLM classifier** — most flexible, handles ambiguous queries. (2) **Keyword router** — fast, rule-based, low cost. (3) **Embedding similarity router** — embed query, compare to index descriptions. (4) **Multi-index fusion** — query all relevant indices, merge and rerank results.

■ Example:

Multi-index RAG for an enterprise AI assistant:

Indices:

products_index: product specs, pricing, features
policies_index: HR, legal, compliance documents
finance_index: reports, forecasts, budgets
technical_index: engineering docs, APIs, runbooks
news_index: daily-updated industry news

Router implementation:

```
router_prompt = '''
Classify this query into one or more categories:
[products, policies, finance, technical, news, multiple]
Query: {query}
Return JSON: {"categories": [...], "confidence": X}
'''
```

Query routing examples:

```
'What is the price of Product X?'
-> categories: ['products'], confidence: 0.97
-> Route to: products_index only
```

```
'What is our parental leave policy?'
-> categories: ['policies'], confidence: 0.95
-> Route to: policies_index only
```

```
'How does our product pricing compare to
industry trends this week?'
-> categories: ['products', 'news'], confidence: 0.89
-> Route to: BOTH products_index + news_index
-> Merge and rerank results from both
-> Generate synthesized answer
```

```
'What are the API specs and current uptime?'
-> categories: ['technical'], confidence: 0.91
-> Route to: technical_index
-> Access control check: user has engineering access? YES
-> Return technical details
```

■ *Interview Tip: Mention access control in multi-index RAG. 'Multi-index RAG naturally enables access control — a sales rep routes to products and news indices, a finance team member routes to finance index, engineers access technical index. The router enforces access policies at query time, not just at document ingestion.'* Shows security thinking.

Production & Evaluation

Q9

How do you evaluate Agentic RAG systems beyond standard RAGAS metrics?

Answer:

Standard RAGAS metrics (faithfulness, relevancy, context precision) measure single-turn RAG quality. Agentic RAG needs additional evaluation dimensions: (1) **Decision quality** — did the agent make the RIGHT retrieval decisions? Did it retrieve when needed? Did it skip retrieval for known facts? Did it correctly identify when to web search? (2) **Trajectory efficiency** — how many steps did the agent take? Fewer steps = better (same result in 3 steps vs 8 steps). (3) **Self-correction accuracy** — when the agent flagged low quality and retried, did the retry actually improve quality? (4) **Tool selection accuracy** — when multiple retrieval tools available, did the agent pick the right one? (5) **Failure mode detection** — did the agent correctly identify when it couldn't answer vs hallucinating a confident wrong answer? Evaluation approach: build a golden dataset with annotated agent trajectories showing the CORRECT decision at each step.

■ **Example:**

Agentic RAG evaluation framework:

Golden dataset: 200 queries with annotated correct trajectories

Evaluation metrics:

1. Decision Quality Score

For each query, did agent make correct retrieve/skip decision?

Score: $\text{correct_decisions} / \text{total_decisions}$

Our system: 91% decision accuracy

2. Trajectory Efficiency

Avg steps taken: 4.2

Avg optimal steps (human annotated): 3.1

Efficiency ratio: $3.1/4.2 = 73.8\%$

(30% more steps than needed -> optimize)

3. Self-Correction Effectiveness

Times agent triggered self-correction: 47

Times correction improved quality: 41 (87%)

Times correction made it worse: 6 (13%) <- investigate!

4. Failure Honesty Rate

Queries agent couldn't answer: 23

Times said 'I don't know' correctly: 19 (83%)

Times hallucinated instead: 4 (17%) <- dangerous!

5. Tool Selection Accuracy

Correct tool selected: 88%

Wrong tool (vector when should use web): 12%

Weekly report dashboard:

Decision quality: 91% (target: >90%) GREEN

Trajectory efficiency: 73.8% (target: >80%) RED

Self-correction effectiveness: 87% GREEN

Failure honesty: 83% (target: >95%) RED <- critical fix needed

■ *Interview Tip: Highlight 'failure honesty rate' as the most critical metric. 'The most dangerous Agentic RAG failure is confident hallucination — the agent says it found an answer when it actually made one up. I track failure honesty rate specifically: what % of the time does the agent correctly say it does not know vs hallucinate. This metric is more important than any RAGAS score for production safety.'*

Q10

How do you handle knowledge conflicts in Agentic RAG when retrieved documents contradict each other?

Answer:

Knowledge conflicts occur when retrieved chunks contain contradictory information — common when documents span different time periods, authors, or update cycles. Four conflict types: (1) **Temporal conflicts** — older doc says price is \$99, newer doc says \$149. (2) **Source authority conflicts** — unofficial FAQ contradicts official policy document. (3) **Version conflicts** — API v1 docs vs v2 docs for

same endpoint. (4) **Opinion conflicts** — different analysts give different market forecasts. Handling strategies: (1) **Recency preference** — always prefer more recent document. Store and check document dates. (2) **Authority ranking** — assign trust scores to sources (official policy > FAQ > blog post). (3) **Conflict detection** — use LLM to explicitly detect contradictions before generating. (4) **Present both** — for genuine disagreements, present both views with attribution. (5) **Escalate to human** — for high-stakes conflicts (medical, legal), don't guess — ask human to resolve.

■ Example:

Conflict detection and resolution pipeline:

User: 'What is the enterprise pricing for Product X?'

Retrieved chunks:

Chunk 1 (pricing_2025.pdf, Jan 2025):

'Enterprise tier: \$5,000/month'

Chunk 2 (pricing_2026.pdf, Mar 2026):

'Enterprise tier: \$7,500/month'

Chunk 3 (sales_faq.md, Feb 2026):

'Enterprise starts at \$6,000/month'

Conflict detection step:

prompt = 'Do these chunks contain contradictory information?

List any conflicts found.'

LLM detects: 'Three different prices found:

\$5,000 (2025), \$7,500 (2026 official),

\$6,000 (2026 FAQ)'

Conflict resolution:

Strategy: Recency + Authority ranking

- Chunk 2 is most recent AND official doc -> highest trust
- Chunk 3 is recent but unofficial (FAQ) -> medium trust
- Chunk 1 is outdated -> lowest trust

Generated answer:

'Enterprise pricing is \$7,500/month as of March 2026 (per official pricing document). Note: some older sources show different prices. Please confirm with your sales representative for current pricing.'

Transparent about conflict + cites authoritative source

Does NOT pick wrong price and present confidently

■ *Interview Tip: Say 'Conflict detection is a separate step I add before generation whenever multiple chunks are retrieved. I explicitly ask the LLM to identify contradictions rather than hoping it resolves them implicitly. Implicit conflict resolution leads to unpredictable behavior — the LLM might pick the wrong source or blend contradictory information into a confused answer.'*

You Made It! ■■

10 Agentic RAG Interview Questions — Complete.

CRAG | Self-RAG | Adaptive | Reflection | GraphRAG | Multi-Index | 2026

■ github.com/amanailab/Production-level-Generative-AI-Interview-Questions

■ 30 RAG Questions | 30 Agentic AI | 30 AI Engineer | 30 AI Agent

■ 30 LLM | 30 SQL | 21 Python | 21 MLOps | 21 LLMOps | 50 ML

amanailab.com

Built with ♥ by AmanAI Lab | 2026